ENSEIRB-MATMECA 1ère année Informatique

Projet Royal Dédale

Rapport de projet

Groupe I1 P5 - Équipe n°8
Ilan Vargas & Achraf Safi

18 décembre 2025



Table des matières

Introduction & Architecture **2**

1 Fondation **3**
1.1 Code source . . . 3
1.1.1 directions . . . 3
1.1.2 positions . . . 3
1.1.3 ressources . . . 3
1.1.4 poches . . . 4
1.2 Tests et validation . . . 4
1.2.1 Tests sur les directions et les ressources . . . 4
1.2.2 Tests sur les positions . . . 5
1.2.3 Tests sur les poches . . . 5
1.2.4 Conclusion sur les tests . . . 5
Conclusion . . . 5

2 Implémentation des tuiles **6**

3 Implémentation du jeu **6**
3.1 Boucle du jeu . . . 6
3.2 Exécution . . . 6
3.2.1 Le Makefile . . . 6
3.2.2 Paramètres d'exécution . . . 6
3.3 Visualisation graphique : TUI . . . 7

4 Limites et améliorations **7**

Introduction

Pb :

Comment concevoir et implémenter une architecture de programmation robuste et extensible permettant de générer, explorer et manipuler un manoir tout en garantissant une gestion correcte des ressources, des interactions entre tuiles et des contraintes imposées par les interfaces abstraites du projet ?

Architecture du programme

Nous détaillerons l'architecture du programme C de notre version de initiale de *Royal Dédale*

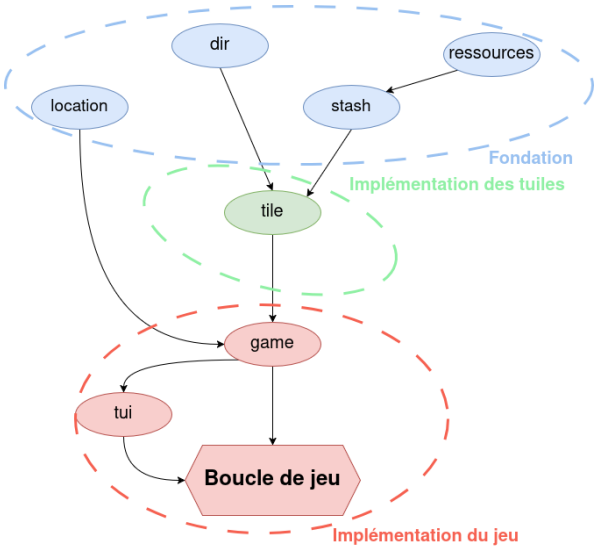


FIGURE 1 – Architecture de la version initiale de notre projet

La figure 1 nous montre les modules de nos programme et leurs interdépendances, chaque flèche indique que le fichier au bout de la flèche est dépendant de celui à l'origine mais aussi de tout les modules dont ce dernier en dépend. Nous pouvons regrouper les modules en trois groupes :

- La **fondation**. Où on implémente les "types" (structures et énumérations) fondamentales pour notre programme : *localisation*, *direction*(*dir*), *ressource* et *stash*(*poche*) ; et leurs opération associées. Ces conceptes servent de base à la quel s'appuie les modules suivents.
- L'**implémentation des tuiles**. Avec les modules précédents nous créeons les tuiles et leurs opérations. C'est de ça qui est composé le manoire implémenter par la suite et une mécanique fondamentale de la partie.
- L'**implémentation du jeu**. C'est la que nous créons le manoir, les fonctions pour le manipuler, des fonction pour l'afficher en forme d'interface textuelle et combinons tout ça dans une boucle qui nous permettra de jouer le jeu. Dans cette étape nous combinons tout notre code précédant pour créé finalment le jeu.

Cet architecture donne un jeu fonctionnel mais forcément de la manière la plus optimal ou complet en terme de jouabilité, nous verrons les limites et les amélioratoins proposé à ce programme. Mais avant ça, nous commençons par detailler la fondation de cette architecture.

1 Fondation

Nous detaillons les "types", et leurs opérations, utilisé comme base de ce projet.

1.1 Code source

1.1.1 directions

On défini un type enuméré dir_t afin de gérer les différentes directions cardinales (nord, sud, west, oest) d'une manière simple et claire tout au long du project.

Le module implémente les fonction `dir_to_string` qui représente une direction par la chaîne de caractère qui lui est associée, ainsi que la fonction `dir_to_short_string` qui renvoie la représentation d'une direction par une petite chaîne de caractère.

De plus, il implémente la fonction `dir_opposite` qui permet d'obtenir la direction opposée à celle fournie en entrée.

1.1.2 positions

On définie la position par une structure $location_t$ contenant un couple de coordonnées (x, y) correspondant respectivement à l'abscisse et l'ordonnée d'une position dans le manoir. Une fonction `location` est fournie ainsi afin de générer cette structure à partir du couple de coordonnées fournies.

Le module inclut les fonctions nécessaires pour manipuler les positions dans le manoir :

— `location_is_valid`, verifie si une position est bien dans le manoir (dans les limites).

— `location_equal`, verifie si deux position a et b sont égales.

— `location_is_adjacent`, pour une position a verifie si la position b est adjacente.

Les deux dernière renvoient faux si une des position n'est pas valide.

1.1.3 ressources

Cet interface permet de définir le type énuméré $resource_t$ qui représente l'ensemble des types de ressources dans le jeu dont le nombre est fixé à $max_resources = 10$, cette implémentation simplifie parsuite le stockage et la manipulation des ressources du joueur dans une poche, décrite dans la section suivante.

L'implémentation inclut des fonctions qui permettent de représenter la ressource en chaine de caractère, soit `resource_to_string` pour le nom complet de la ressource et `resource_to_short_string` pour une représentation minimale (généralement un caractère).

L'implémentation de ces fonctions est réalisée grâce à la structure $switch-case$ afin de gérer le cas de toutes les ressources.

1.1.4 poches

Après avoir défini les types de ressources via une énumération, nous implémentons maintenant le module *stash* (poche) qui constitue le système gestion des ressources du joueur.

On introduit la structure $stash_t$, qui représente une poche de ressources. Il s'agit d'un tableau dont la taille correspond exactement au nombre de types de ressources en jeu où chaque case du tableau stocke la quantité de la ressource associer à son indice, selon $resource_t$. Après avoir implémenter les ressources dans le module précédant, on implémente un module

Ce fichier regroupe l'ensemble des opérations sur les poches :

- `stash_add`, l'addition de deux poches a et b : retourner une poche contenant pour chaque ressource l'addition du nombre de cette ressource dans a et b
- `stash_sub`,la soustraction d'une poche a à une autre b, pour chaque ressource de a on la soustrait à celle de b (0 si la valeur est négative)
- `stash_contains`, verifie si une poche b est contenue dans une poche a (pour toutes les ressources de b, ça quantiter est plus petite ou égale à celle de a ; deux poches égales ce sontiennent)

On a une fonction supplémentaire pour une représentation en chaine de caractère de la poche `stash_to_string`, utile pour une interface du jeu.

1.2 Tests et validation

Dans cette partie on s'intéressera à l'implémentation des tests et comment ils permettent de valider la correction des fonctions. Généralement, les fonctions du code source sont testées sur plusieurs entrées ce qui demande la création de trois tableaux, un tableau pour les entrées, un pour les sorties et un autre contenant les solutions attendues, pour comparer la sortie à la solution. Un exemple de ces tests sera fourni dans ce qui suit.

1.2.1 Tests sur les directions et les ressources

Les tests sur ces deux modules servent à vérifier la sortie des deux fonctions de conversion des directions et ressources en leurs noms sous forme de chaine de caractère, donc la validation des tests correspond à la verification de l'égalité entre la chaine de sortie et celle attendu.

D'autre part, le test `dir_oposite` demeure simple grâce au type énuméré des directions, il suffit de vérifier l'égalité entre la sortie et la solution autant que deux entiers.

```
TESTS ON resource.c
Tests on resource_to_string
PASSED  input:   0   output: NOTHING
PASSED  input:   1   output: STEP
PASSED  input:   5   output: SILK
PASSED  input:   9   output: OIL
PASSED  input:  10   output: CRYSTAL
Tests on resource_to_short_string
PASSED  input:   0   output: N
PASSED  input:   1   output: St
PASSED  input:   4   output: Gd
PASSED  input:  10   output: C
```

Code 1 – Exemple des tests sur les ressources

1.2.2 Tests sur les positions

Afin de valider les tests sur les fonctions manipulant les positions du manoir, appelons les fonction sur des positions differentes dans le but de tester tout les cas pertinents, notamment les postions dedans et dehors le manoire pour tester la validité de la position du manoir ; des couples égaux adjacents ou pas pour les fonctions qui comparent deux positions

1.2.3 Tests sur les poches

Avant de procéder aux tests, une fonction est implémentée afin de générer des poches dont les ressources sont des valeurs aléatoires inférieures à 10. Cette fonction permettera ensuite d'observer facilement les quantités de chaque ressource de la poche retournée en les comparant à celles des poches passées en argument, et ainsi de déduire si les opérations effectuées correspondent bien aux opérations élémentaires appliquées ressource par ressource.

1.2.4 Conclusion sur la validité

Même si nous ne pouvons pas tester toutes les combinaisons posibles pour chaque fonction, on deduit qu'on a tester assez de cas pertinents differents pour assumer les fonctions valide et poser notre programme dessu.

Conclusion

L'ensemble des fichiers du code source présentés dans cette section ne présente pas de difficultés particulières en termes d'implémentation ou de complexité mais ils sont

toutefois essentiels, car ils permettent de définir et de structurer les données utilisées par la suite. Le découpage du code en plusieurs fichiers, associé à la compilation séparée, facilite le travail en équipe. Chaque module peut être développé, modifié ou amélioré indépendamment des autres, tout en respectant les interfaces définies par les fichiers d'en-tête. Cette organisation permet ainsi d'intégrer différentes implémentations d'un même module sans impacter le reste du programme.

Nous verrons maintenant comment passer à l'étape suivante l'implémentation des tuiles.

2 Implémentation des tuiles

3 Implémentation du jeu

3.1 Boucle du jeu

Implémentation

Interprétation

La boucle du jeu consiste à vérifier, à chaque itération, que le joueur n'a pas dépassé le nombre de tours possibles, qu'il n'a pas gagné la partie et qu'il a toujours des ressources. Chaque tour du jeu consiste à avoir deux tableaux de positions adjacentes, un tableau pour les positions dont les tuiles sont encore vides et un autre tableau pour les tuiles qui sont déja ouvertes. Selon la décision du joueur, s'il choisit de placer un tuile, on prend une position valide parmi celles des tuiles vides et on la place. Sinon, le joueur est censé à ce déplacer, il suffit alors de choisir une position parmi celles des tuiles déja ouvertes et le joueur s'y déplace, l'ensemble des opérations réalisées pendant le jeu repose sur la structure *game_state* définit dans la figure.

3.2 Exécution

3.2.1 Le Makefile

3.2.2 Paramètres d'exécution

L'analyse de la ligne de commande pendant l'exécution se fait au moyen de la bibliothèque getopt, cela permet pour un même exécutable de changer les paramètres essentiels suivants

— initialiser la graine du générateur aléatoire : -s
— passer l'argument de la fonction init_tiles : -t
— fixer le nombre maximum de tours de jeu possible : -n

D'autres paramètres ont été envisagés afin de donner du sens au mot 'jeu'

- rendre le jeu interactif afin de pouvoir jouer : -i
- permettre la visualisation graphique de manoir : -g

3.3 Visualisation graphique : TUI

4 Limites et améliorations

Conclusion

```c
while (current_turn <= n && !is_player_won && game_state.steps.r[STEP] > 0)
{
    printf("\e[1mTurn number %d out of %d\e[0m\n", current_turn, n);
    int num_possible_locations = list_empty_locations_with_finite_opening_cost(possible_locations, &game_state);
    if (current_turn == 1 || (num_possible_locations && decide_to_place_tile(is_iterative)))
    {
        printf("Place your tile\n");
        struct location_t selected_location = select_location(is_interactive, possible_locations, num_possible_locations);
        int num_available_tiles = generate_some_tiles(selected_location, available_tiles, &game_state);
        struct tile_t* selected_tile = select_tile(is_interactive, available_tiles, num_available_tiles);
        place_tile(selected_location, selected_tile, &game_state);
        tui_place_tile(is_graphic, selected_location, selected_tile);
    }
    else
    {
        printf("Move the player\n");
        int num_possible_locations = list_open_location(possible_locations, &game_state);
        struct location_t selected_location = select_location(is_interactive, possible_locations, num_possible_locations);
        tui_move_player(is_graphic, selected_location, &game_state);
        move_player(selected_location, &game_state);
    }
    current_turn++;
    tui_print(is_graphic);
    is_player_won = exit_reached(&game_state);
}
```

Code 2 – Implémentation de la boucle du jeu